AI x API：打造你的 LINE 記帳管家

課程啟動：我們今天要做什麼？

1. 成品展示 (Demo)
    - **情境模擬**：傳送「今天午餐在巷口麵店花了 120 元」給機器人。
    - **AI 運作**：機器人自動解析出 {"品項": "午餐", "金額": 120}。
    - **智慧分析**：輸入「分析」，機器人會總結花費並給出毒舌或溫柔的理財建議。

課程啟動：我們今天要做什麼？

2. 三大核心工具簡介

- **LINE Messaging API**：機器人的「五官」，負責接收與傳送訊息。
- **Gemini AI**：機器人的「大腦」，負責讀懂人類隨性的講話方式。
- **Google APP Scripts**：機器人的「骨架與靈魂」，負責串接 API、計算金額與存放資料。

上半場課程導覽

- 變數、資料型態與運算 — *我的第一行程式*
- 資料結構：清單與字典 — *數位小帳本的雛型*
- 函式與字串處理 — *讓程式學會思考*
- 用Python串接第一個API
- 程式碼

變數、資料型態與運算

1. Hello Python!

- **開發環境**：使用 Google Colab (無需安裝，打開瀏覽器即可寫程式)。
- **第一行程式碼**：

```
print("你好，記帳管家已上線！")
```

變數、資料型態與運算

2. 變數與標籤

把變數想像成一個「貼了標籤的盒子」，用來儲存記帳資料：

- **字串 (String)**：儲存品項名稱，例如 "咖啡"。
- **整數 (Integer)**：儲存金額，例如 60。
- **浮點數 (Float)**：儲存有小數點的金額，例如 30.5。

```
item = "早餐"
price = 100
```

變數、資料型態與運算

3. 基礎運算

計算總支出，這就是記帳程式的核心：

```
breakfast = 45
lunch = 120
total = breakfast + lunch
print("今天目前的總花費是：", total)
```

實作練習 1：餐費計算機

任務：VibeThinking系統

小明想要記錄自己一天的伙食開銷。

請撰寫一個程式，讓使用者依序輸入早餐、午餐及晚餐的花費（皆為正整數），

並計算出這三項花費的總金額與平均每項花費。

資料結構 (List & Dictionary)

1. 清單 (List) — 儲存多筆支出

當你有一連串花費時，不需要幫每個都取名字，直接排隊放進清單：

```
expenses = [100, 50, 200, 60]
# 取得第一筆資料 (索引從 0 開始)
print(expenses[0])
```

資料結構 (List & Dictionary)

字典 (Dictionary)

記帳不只要記金額，還要記「品項」。字典用 Key: Value 的方式儲存：

```
record = {
    "item": "拿鐵咖啡",
    "price": 75,
    "date": "2023-10-27"
}
print(record["item"])
```

迴圈 (Loop)

For 迴圈：自動化加總

讓電腦幫你數錢，不要自己加：

```
total = 0
prices = [100, 250, 80]


for p in prices:
    total = total + p

print(f"清單加總結果：{total}")
```

實作練習 2：每日餐費統計

任務：

小明想要統計一天的餐費。請撰寫一個程式，

讓使用者輸入三筆餐點資料，

每筆資料包含「餐點名稱」與「金額」存入字典中，

並算出總花費。

函式與字串處理

1. 函式 (Function) — 包裝邏輯

把常用的功能包起來，未來只需呼叫名字：

```
def calculate_total(price_list):
  total = 0
  for price in price_list:
    total += price
  return total


my_wallet = [50, 100, 200]
print(calculate_total(my_wallet))
```

函式與字串處理

2. 字串拆解 — 處理使用者輸入

使用者在 LINE 可能會傳「早餐 100」，我們要把它拆開：

```
user_input = "早餐 100"
parts = user_input.split(" ") # 依照空格拆分
item_name = parts[0]
amount = int(parts[1]) # 把文字轉成數字
```

函式與字串處理

3. 條件判斷 (if-else)

讓機器人具備判斷能力：

```
budget = 500
total_spend = 550


if total_spend > budget:
    print("⚠️ 警告：你已經超支了！")
else:
    print("✅ 預算控管良好。")
```

實作練習 3：預算管控

任務：

請寫一個函式名為 budget_control(budget, spending)，

接收兩個整數作為參數 (預算(budget)、花費(spending))。

回傳規則：

若 花費大於預算，請回傳字串 'Over budget'

否則（花費小於或等於預算），請回傳字串 'Under budget'

注意：

function名稱要一模一樣，括號裡的參數名稱可以不一樣，但參數個數一定要一致

用Python串接第一個API

- **情境**：你打開 Uber/Uber Eats，你可以看到地圖、司機位置、餐廳菜單。
- **提問**：Uber 是一間地圖公司嗎？不是。那地圖哪來的？
- **解答**：Uber 跟 Google Maps 借的。怎麼借？透過 API。
- 現代軟體開發不是從零開始造輪子，而是像「樂高積木」一樣，把別人的強大功能（地圖、天氣、金流）透過 API 接過來用。

生活中的 API：為什麼 Uber 會有地圖？

- **情境**：你打開 Uber/Uber Eats，你可以看到地圖、司機位置、餐廳菜單。
- **提問**：Uber 是一間地圖公司嗎？不是。那地圖哪來的？
- **解答**：Uber 跟 Google Maps 借的。怎麼借？透過 API。
- 現代軟體開發不是從零開始造輪子，而是像「樂高積木」一樣，把別人的強大功能（地圖、天氣、金流）透過 API 接過來用。

用餐廳比喻：API 就是那位「服務生」

- **場景**：你 (User) 去一間高檔餐廳吃飯。
- **角色分配**：
  - **你 (Client/User - Python程式)**：想要吃牛排（想要數據）。
  - **廚房 (Server/Database)**：有食材、會做菜（擁有數據、運算能力）。
  - **服務生 (API)**：你不能直接衝進廚房亂拿，你需要透過「服務生」。

用餐廳比喻：API 就是那位「服務生」

1. **點餐 (Request)**：你看著「菜單 (Documentation)」，跟服務生說「我要一份牛排 (GET /steak)」。
2. **傳遞**：服務生把單子送到廚房。
3. **出餐 (Response)**：廚房做好了，服務生把牛排端到你桌上。

- **重點**：你不需要知道廚房瓦斯爐怎麼開、肉怎麼切，你只要會「呼叫服務生」就好。

API 講什麼話？認識 JSON

- 服務生端回來的不是真的盤子，通常是一串文字，我們稱為 **JSON** (JavaScript Object Notation, .json)。
- Python 裡面的 **Dictionary (字典)** 長得跟它幾乎一模一樣。

```
{
  "status": "success",
  "message": "[https://images.dog.ceo/breeds/shiba/shiba-
8.jpg](https://images.dog.ceo/breeds/shiba/shiba-8.jpg)"
}
```

動手實作 - Python 隨機狗狗產生器

- 步驟 1：引入幫手 (Import)

```
import requests
import json
import webbrowser
import time
from IPython.display import Image, display
```

動手實作 - Python 隨機狗狗產生器

- 步驟 2：呼叫服務生 (Requests.get)

```
# 設定 API 的網址
url =
"https://dog.ceo/api/breeds/image/random"


print("🐶 正在聯絡狗狗總部 (API)...")


# 1. 發送請求 (Request)
# 這就像是透過電話向餐廳點餐
response = requests.get(url)
```

動手實作 - Python 隨機狗狗產生器

- 步驟 3：解析包裹 (JSON)

```
# 3. 解析 JSON 資料
    # 將回傳的文字轉換成 Python 的字典 (Dictionary)
    data = response.json()

    # 顯示完整的資料結構
    print("\n--- API 回傳的完整內容 (JSON) ---")
    print(json.dumps(data, indent=4)) # indent=4 讓排版漂亮一點
    print("--------------------------------\n")
```

動手實作 - Python 隨機狗狗產生器

- 步驟 4：顯示圖片

```
# 4. 提取我們要的資訊
    # 我們只需要 'message' 欄位裡的圖片網址
    image_url = data['message']

    print(f"🎉 找到狗狗了！網址是：\n{image_url}")

    # 5. 實際打開圖片 (視覺化回饋)
    print("\n📸 圖片顯示如下：")
    display(Image(url=image_url, width=400))

else:
    print(f"❌ 發生錯誤，狀態碼：
{response.status_code}")
```

動手實作 - Python 隨機狗狗產生器

- 完整程式碼

上半場總結

1. 我們學會了如何定義資料 (**變數、清單、字典**)。
2. 我們學會了如何處理資料 (**運算、迴圈**)。
3. 我們學會了如何結構化邏輯 (**函式、判斷式**)。
4. 用 python 串接第一個API

下半場課程導覽

- 建立機器人的身體 — *LINE & GAS連線實作*
- 裝上 AI 的大腦 — *Gemini API 語意解析*
- 功能組裝與個性化 — *理財分析與角色設定*

- *Line Chatbot主程式*

Line Chatbot程式步驟

1. 準備 Google 試算表與 GAS

2. LINE Developers 帳號設定

    2.1. 登入：前往 LINE Developers Console 並使用 LINE 帳號登入。

    2.2. 建立 Provider：點擊「Create」，名稱可取為 AI-Workshop。

    2.3. 建立 Messaging API Channel： (1)類型選擇 Messaging API、(2)填寫必要資訊（名稱、描述、分類）。

Line Chatbot程式步驟

2. LINE Developers 帳號設定

    2.4. 取得重要金鑰： 在「Messaging API」頁面最下方，點擊「Issue」產生 Channel access token

3. 取得 Gemini API Key

4. Apps Script串接Gemini API Key

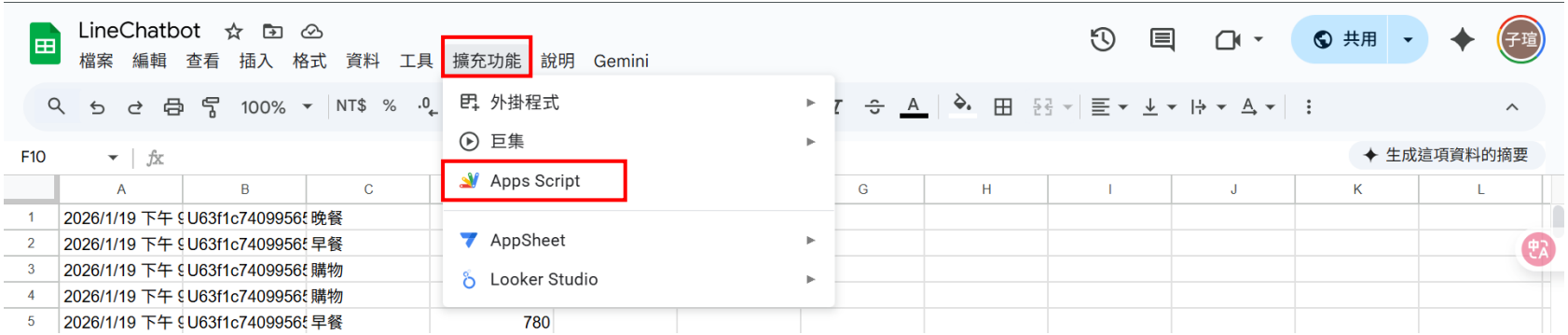
5. 部署程式碼

6. 串接 LINE Webhook

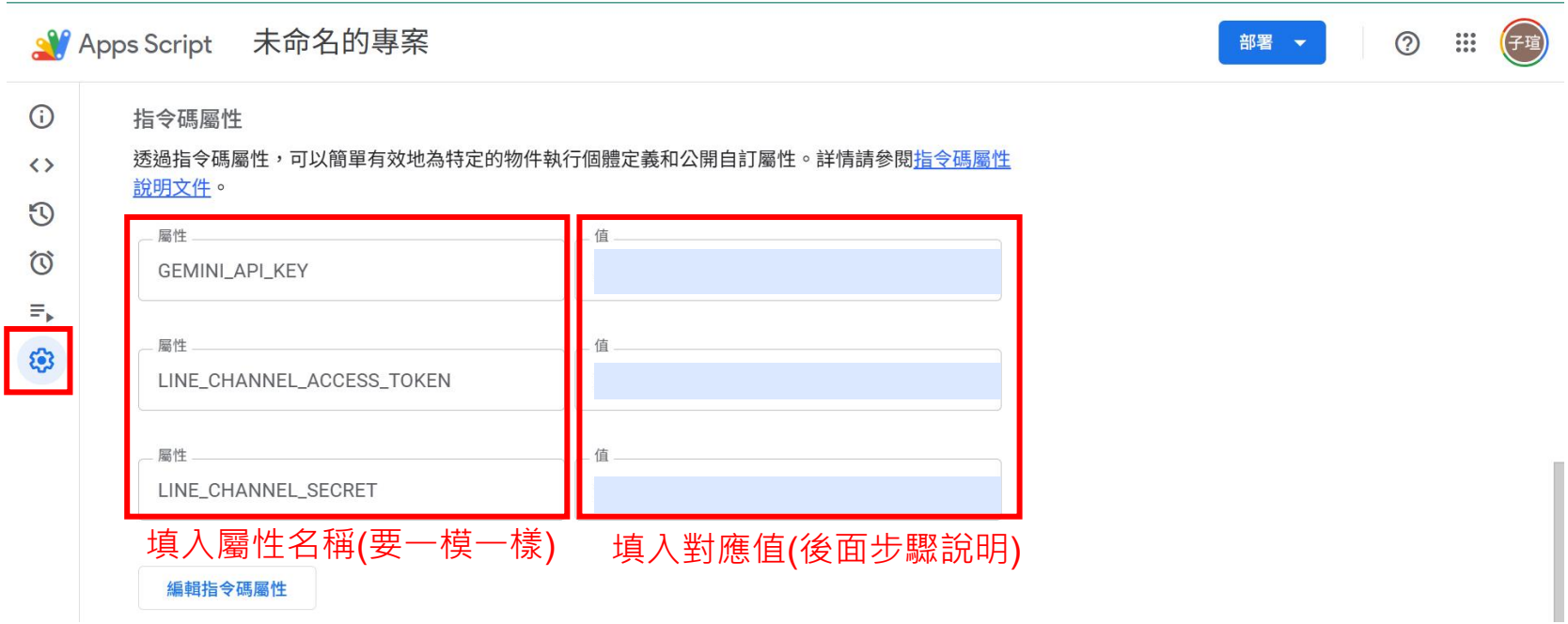
7. LINE 帳號「回應設定」

1. 準備 Google 試算表與 GAS

- **建立試算表**：新建一個 Google 試算表，命名為 “AI 記帳本” 。
- **開啟腳本編輯器**：在試算表中點擊 「**擴充功能**」 > 「**Apps Script**」。



1. 準備 Google 試算表與 GAS 新增指令碼屬性



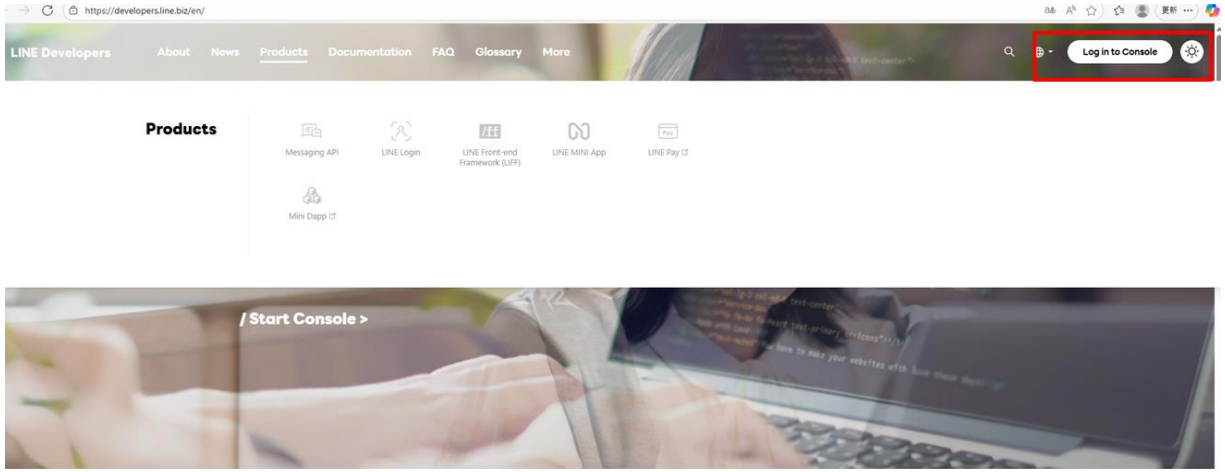
1. 準備 Google 試算表與 GAS 貼上程式碼與部署

- 刪除 Apps Script 編輯器中的所有內容，貼入 code.gs 的內容。
- **初始化試算表**

2.LINE Developers 帳號設定

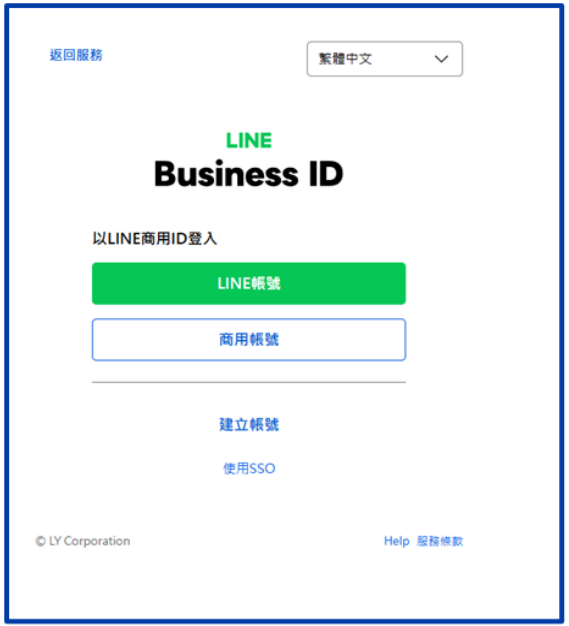
2.1 登入：前往 LINE Developers Console 並使用 LINE 帳號登入。

https://developers.line.biz/en/



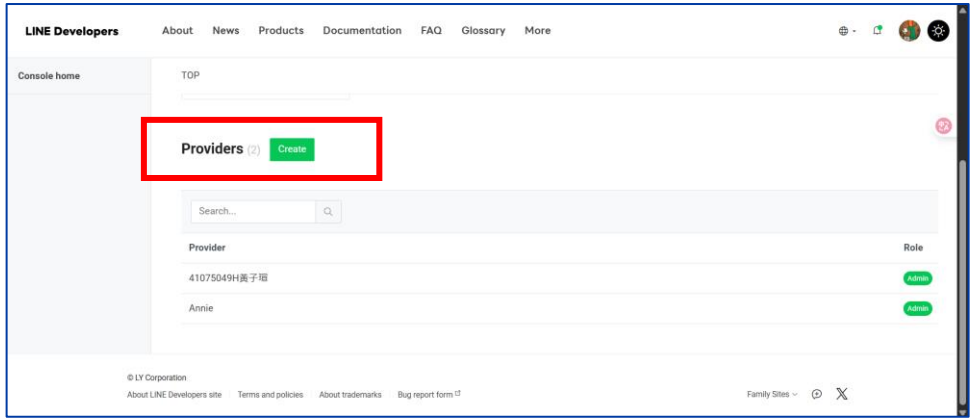
2.LINE Developers 帳號設定

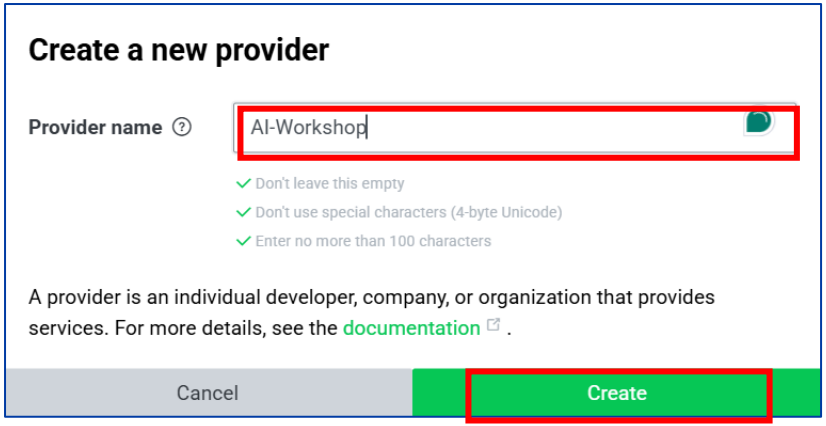
2.1 登入：前往 LINE Developers Console 並使用 LINE 帳號登入。



2.LINE Developers 帳號設定

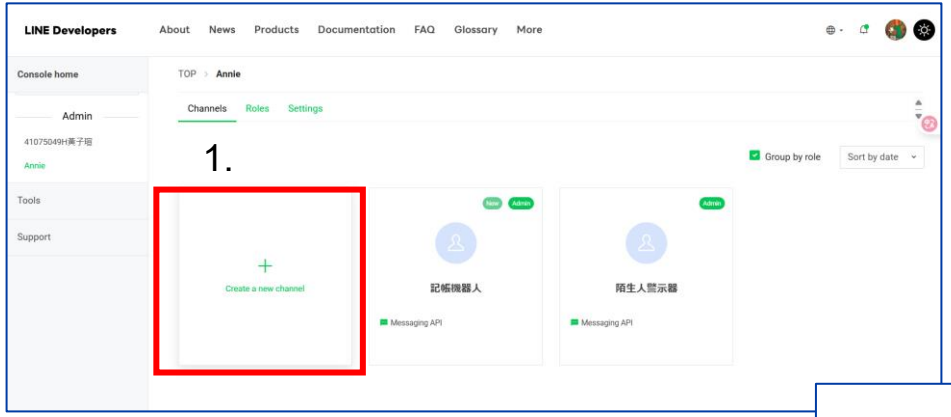
2.2. 建立 Provider：點擊「Create」，名稱可取為 AI-Workshop。

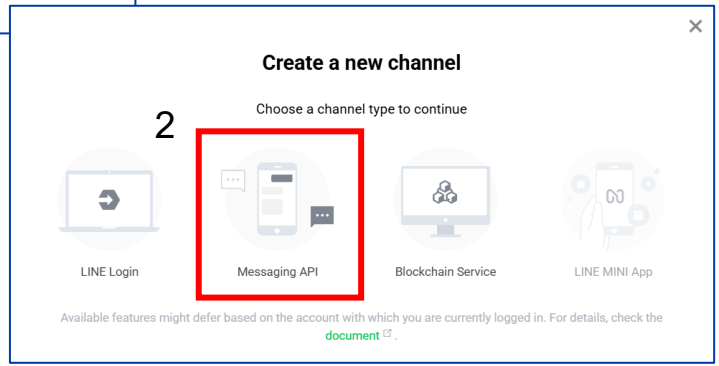




2. LINE Developers 帳號設定

2.3. 建立 Messaging API Channel： (1)類型選擇 Messaging API、(2)填寫必要資訊（名稱、描述、分類）。





2.LINE Developers 帳號設定

2.3.建立 Messaging API Channel： (1)類型選擇 Messaging API、(2)填寫必要資訊（名稱、描述、分類）。

2.LINE Developers 帳號設定

2.3.建立 Messaging API Channel： (1)類型選擇 Messaging API、(2)填寫必要資訊（名稱、描述、分類）。

2.LINE Developers 帳號設定

2.3.建立 Messaging API Channel： (1)類型選擇 Messaging API、(2)填寫必要資訊（名稱、描述、分類）。

2.LINE Developers 帳號設定

2.3.建立 Messaging API Channel： (1)類型選擇 Messaging API、(2)填寫必要資訊（名稱、描述、分類）。

同意我們使用您的資訊

LY Corporation為了完善本公司服務，需使用企業帳號（包括但不限於LINE官方帳號及其相關API產品;以下合稱「企業帳號」）之各類資訊。若欲繼續使用企業帳號，請確認並同意下列事項。

■ 我們將會蒐集與使用的資訊

- 用戶傳送及接收的傳輸內容（包括訊息、網址資訊、影像、影片、貼圖及效果等）。
- 用戶傳送及接收所有內容的發送或撥話格式、次數、時間長度及接收發送對象等（下稱「格式等資訊」）網路電話及視訊通話）及其他功能所處理的內容格式等資訊。
- 企業帳號使用的IP位址、使用各項功能的時間、已接收內容是否已讀、網址的點選等（包括但不限於連結應用程式使用網路瀏覽器及使用時間的紀錄）及LY Corporation隱私權政策所述的其他資訊。

■ 我們蒐集與使用資訊並提供給第三方的目的

LY Corporation將會使用上述資訊於以下目的，至用戶終止企業帳號服務或企業帳號服務被終止為止，這些資訊依據日本《財務報表之用語、格式及編制方法規則》(註)所定義之子公司或關係企業，及日本境外之子公司及委託的服務供應商及分包商。此外，為了提供企業帳號使用相關之支援服務，這些資訊可能與LY Corporation之授與次代理商（經LY Corporation授權代理商授權履行代理業務之代理商）分享。

- 防止未經授權的使用，包括就檢舉資訊進行確認及提供任務，及開發工具以防止未經授權之使用。
- 提供、開發及改善我們的服務，包括考量易於用戶使用的螢幕頁面設計。
- 提供最佳服務，包括呈現LY Corporation所認解用戶偏好的廣告。

若您對本同意書內容有任何問題或意見，請透過聯絡表單與我們聯繫。

如果授予此處同意的人不是企業帳號所有人所授權之人，請事先取得該被授權人的同意。如果LY Corporation接Corporation得中止該企業帳號的使用，且不為因此而生的任何情事負責。

註：本規則以日文版做成，英文版僅供參考，如有任何歧異應以日文版內容為準。

同意

LINE官方帳號使用條款更新啟事
Notice Regarding Updates to LINE Official Account Terms of Use

感謝您使用 LINE 官方帳號。

LINE官方帳號使用條款（下稱「使用條款」）已於2025年2月19日進行更新。此次更新，係為明示台灣連線股份有限公司（下稱「本公司」），依照台灣法律規定，得將使用條款內所定之台灣地區客戶資料，以使用條款及本公司隱私權政策所示之目的、期間、地區、對象及方式為蒐集、處理及利用。
新版LINE官方帳號使用條款請見 https://terms2.line.me/official_account_terms_tw。
若您同意以上使用條款更新，請點選以下「了解並繼續使用」之按鈕。

非常感謝您支持LINE官方帳號。

Thank you for using LINE Official Account.

LINE Official Account Terms of Use (the "TOU") has been updated on February 19, 2025. The purpose of this update is to expressly state that LINE Taiwan Limited (the "Company") may, in accordance with the laws of Republic of China (Taiwan), collect, process, and utilize the information of its customers in Taiwan as set forth in the TOU for the purposes, during the period of time, in the areas, by the subjects, and in the manner as indicated in the TOU and the Company's Privacy Policy.
The updated LINE Official Account Terms of Use can be found at https://terms2.line.me/official_account_terms_tw.

Thank you very much for your support of LINE Official Account.

了解並繼續使用
I understand and want to proceed

2.LINE Developers 帳號設定

2.3.建立 Messaging API Channel： (1)類型選擇 Messaging API、(2)填寫必要資訊（名稱、描述、分類）。

2.LINE Developers 帳號設定

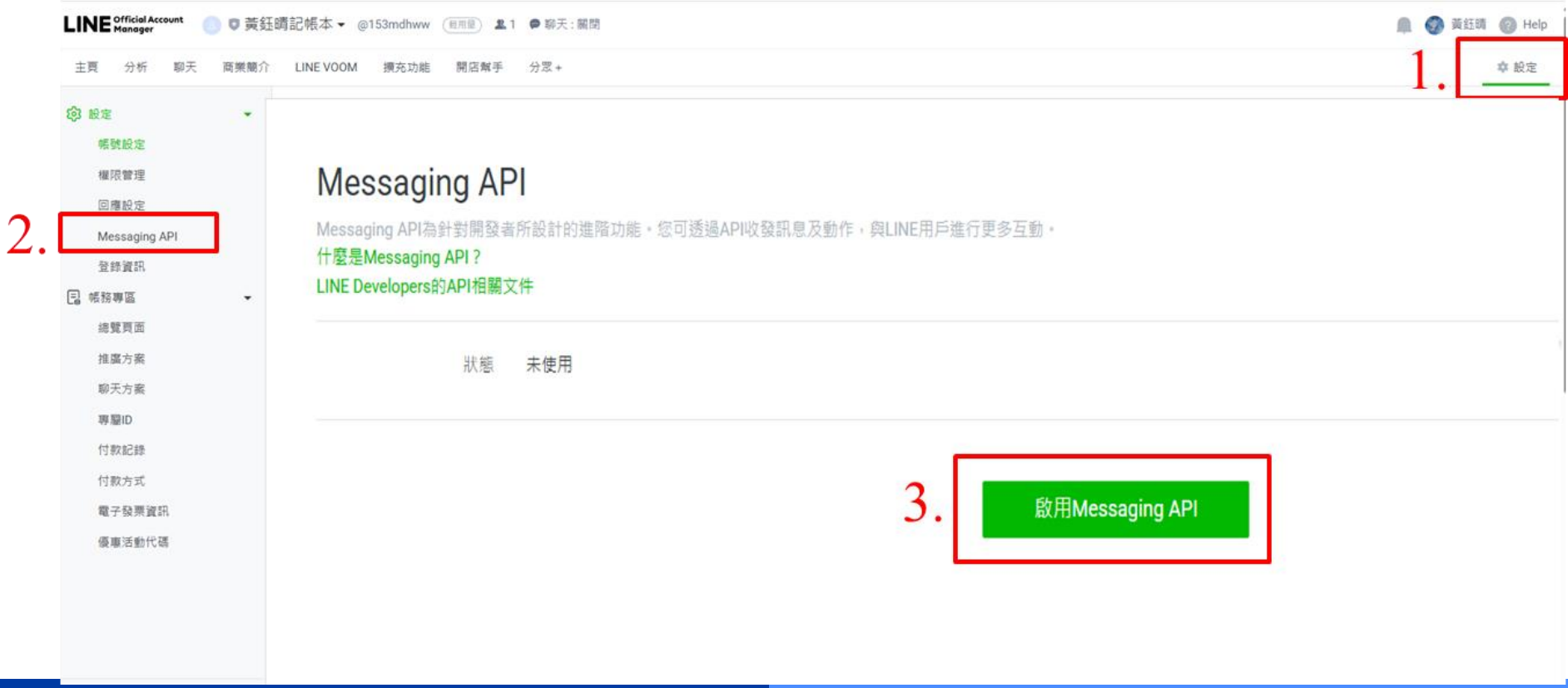
2.3.建立 Messaging API Channel： (1)類型選擇 Messaging API、(2)填寫必要資訊（名稱、描述、分類）。

2.LINE Developers 帳號設定

2.3.建立 Messaging API Channel： (1)類型選擇 Messaging API、(2)填寫必要資訊（名稱、描述、分類）。

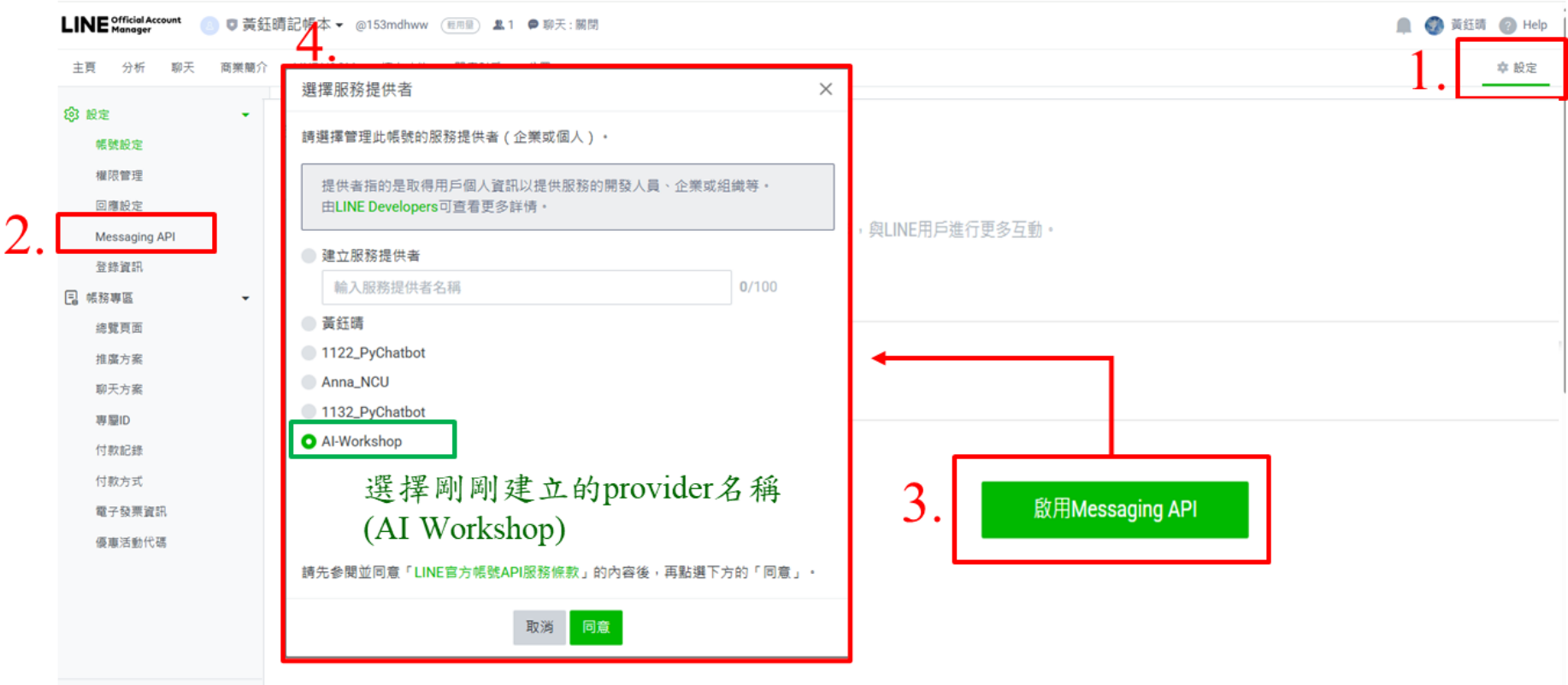
2.LINE Developers 帳號設定

2.3.建立 Messaging API Channel： (1)類型選擇 Messaging API、(2)填寫必要資訊（名稱、描述、分類）。



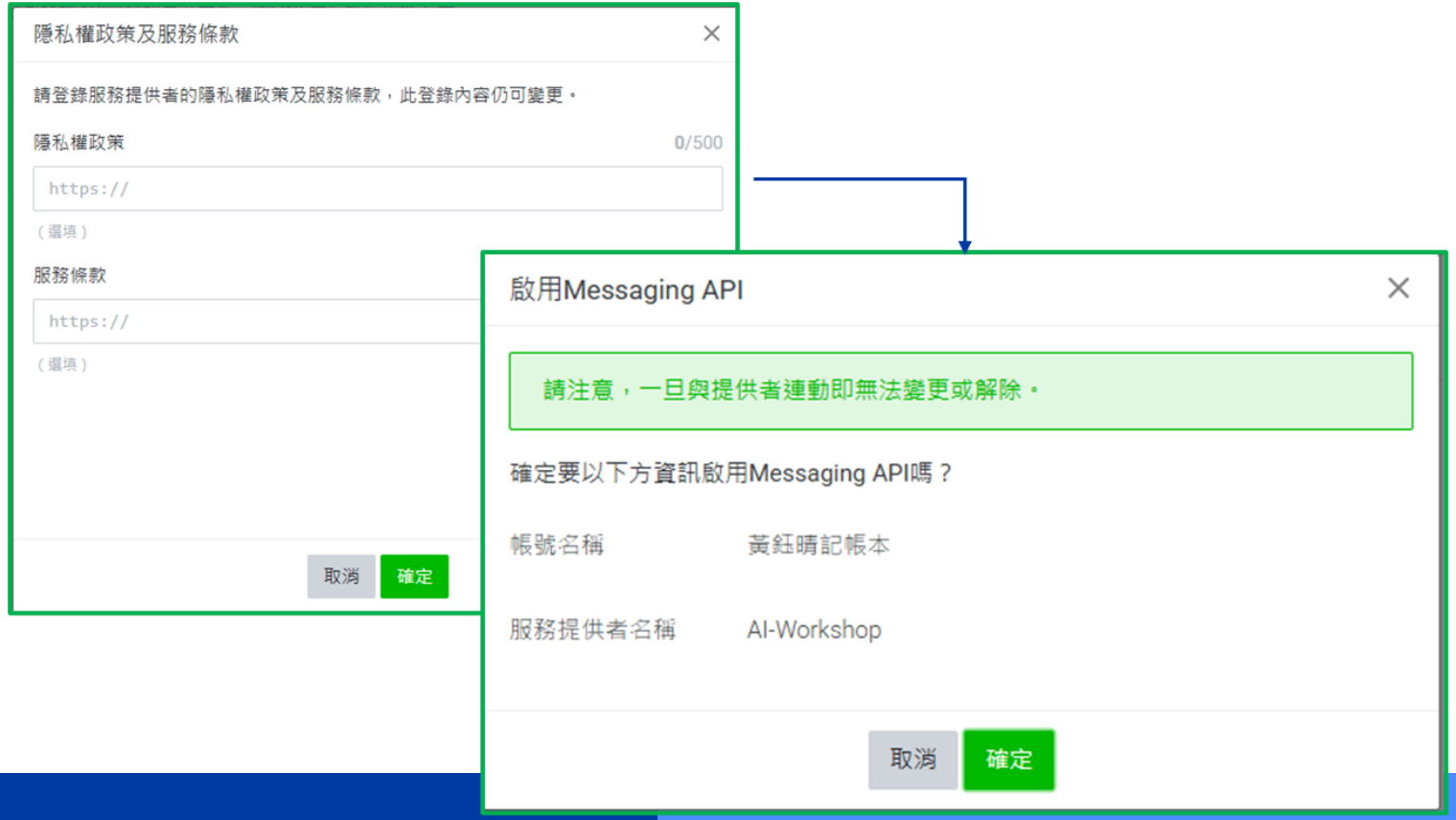
2. LINE Developers 帳號設定

2.3. 建立 Messaging API Channel： (1)類型選擇 Messaging API、(2)填寫必要資訊（名稱、描述、分類）。



2. LINE Developers 帳號設定

2.3. 建立 Messaging API Channel： (1)類型選擇 Messaging API、(2)填寫必要資訊（名稱、描述、分類）。

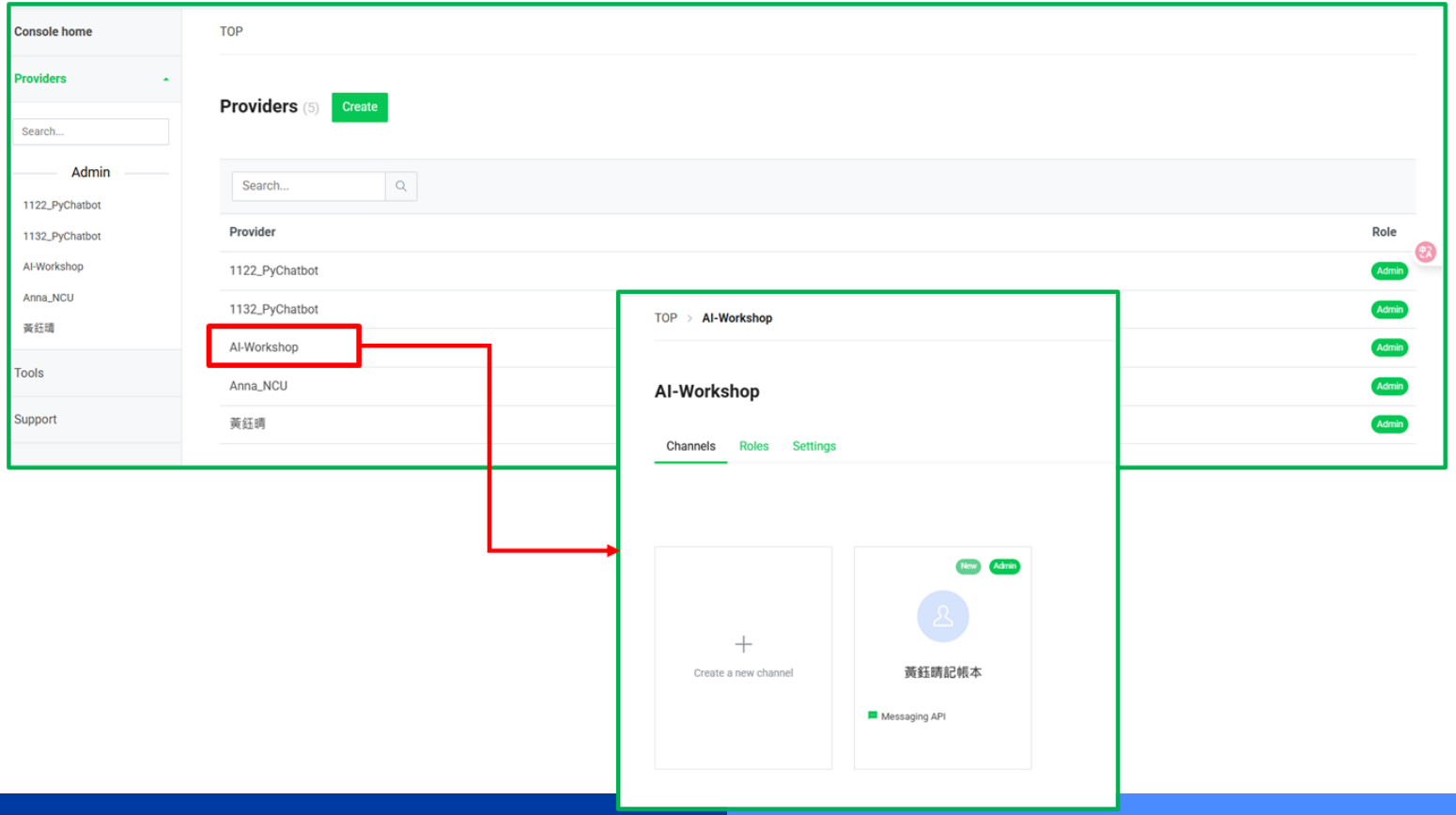


2. LINE Developers 帳號設定

2.3. 建立 Messaging API Channel： (1)類型選擇 Messaging API、(2)填寫必要資訊（名稱、描述、分類）

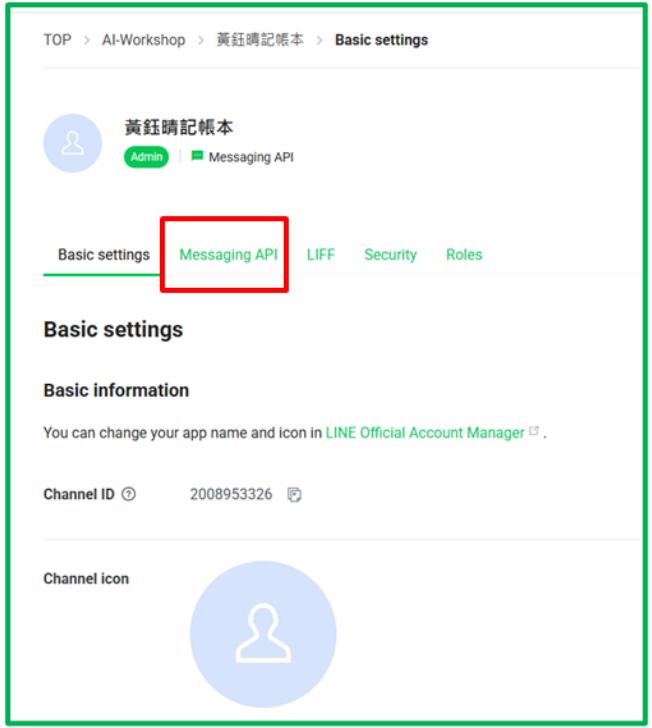
2. LINE Developers 帳號設定

2.3. 建立 Messaging API Channel： (1)類型選擇 Messaging API、(2)填寫必要資訊（名稱、描述、分類）。



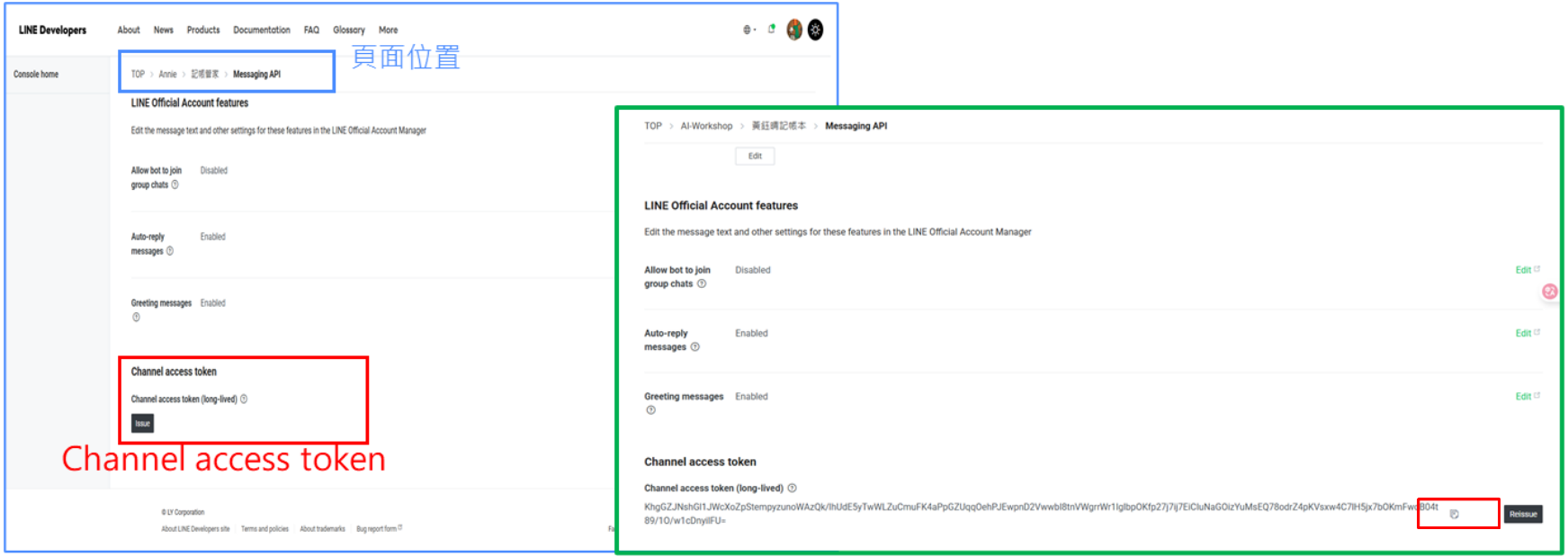
2. LINE Developers 帳號設定

2.3.建立 Messaging API Channel： (1)類型選擇 Messaging API、(2)填寫必要資訊（名稱、描述、分類）。



2. LINE Developers 帳號設定

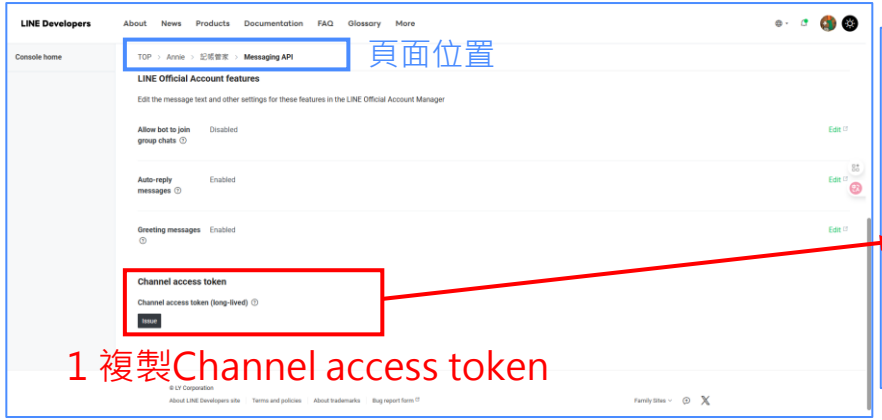
2.4. 取得重要金鑰：在「Messaging API」頁面最下方，點擊「Issue」產生Channel access token



複製所產生的Channel access token

2.LINE Developers 帳號設定

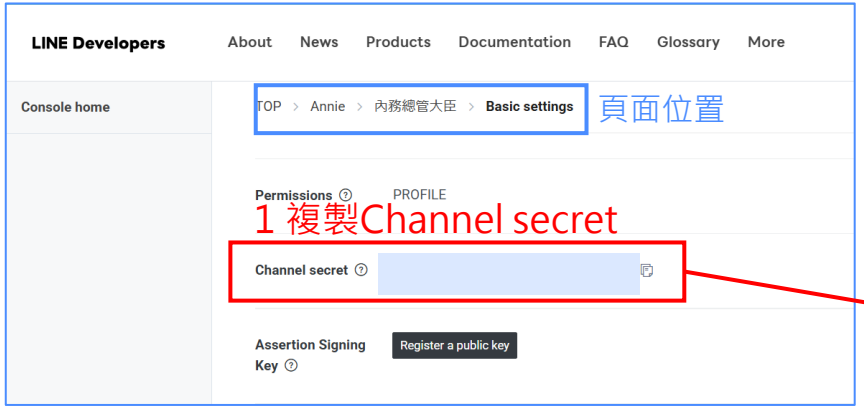
2.4.取得重要金鑰：在「Messaging API」頁面最下方，點擊「Issue」產生Channel access token



LINE_CHANNEL_ACCESS_TOKEN

2. LINE Developers 帳號設定

2.4.取得重要金鑰：在「Messaging API」頁面最下方，點擊「Issue」產生Channel access token

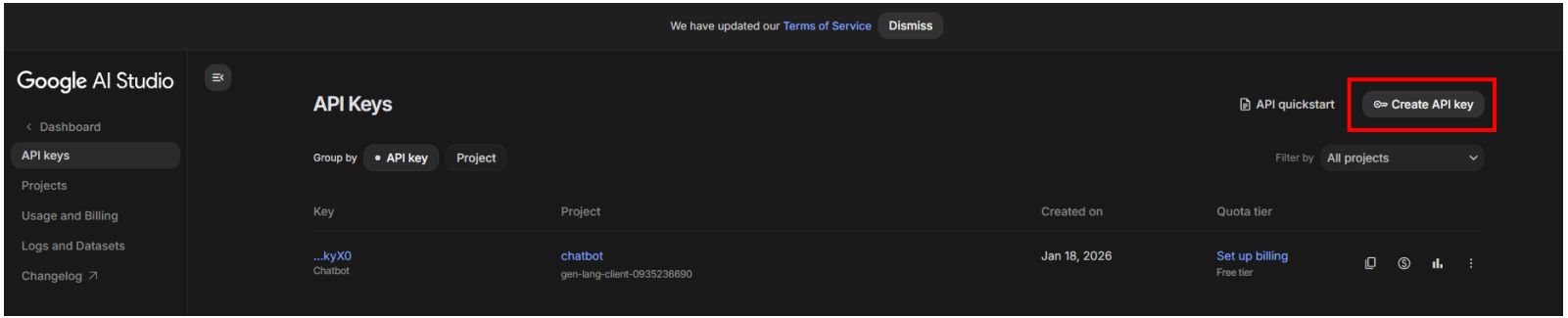


LINE_CHANNEL_SECRET

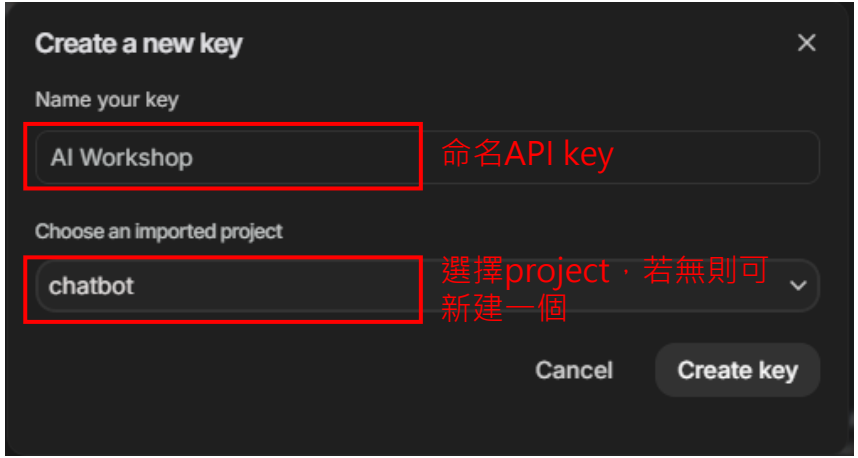
3. 取得 Gemini API Key

3.1 創建Gemini API Key

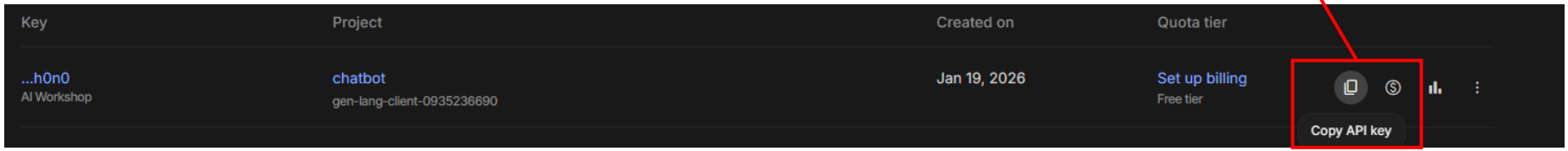
- 前往 Google AI Studio。
- 點擊左側的 **Get API key**。
- 建立一個新的 API Key 並複製（這把鑰匙讓你的機器人擁有 AI 的大腦）。



4. Apps Script串接Gemini API Key

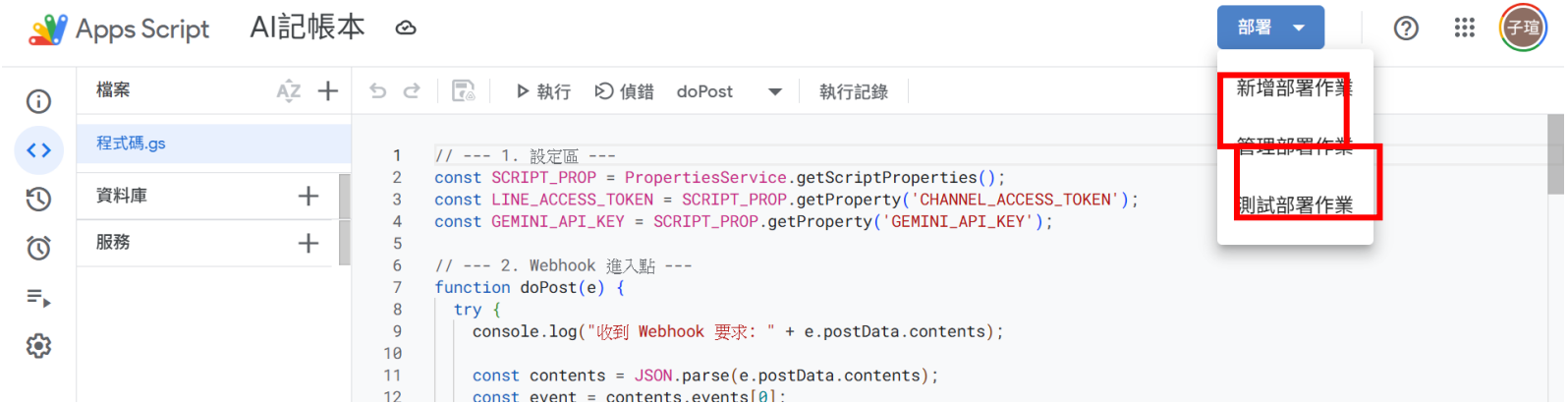


GEMINI_API_KEY



按此可複製API key

5.部署網頁程式碼


Apps Script
AI記帳本
部署
子瑄
檔案
執行
偵錯
doPost
執行記錄
新增部署作業
管理部署作業
測試部署作業
程式碼.gs
資料庫
服務
// --- 1. 設定區 ---
const SCRIPT_PROP = PropertiesService.getScriptProperties();
const LINE_ACCESS_TOKEN = SCRIPT_PROP.getProperty('CHANNEL_ACCESS_TOKEN');
const GEMINI_API_KEY = SCRIPT_PROP.getProperty('GEMINI_API_KEY');
// --- 2. Webhook 進入點 ---
function doPost(e) {
try {
console.log("收到 Webhook 要求: " + e.postData.contents);
const contents = JSON.parse(e.postData.contents);
const event = contents.events[0];

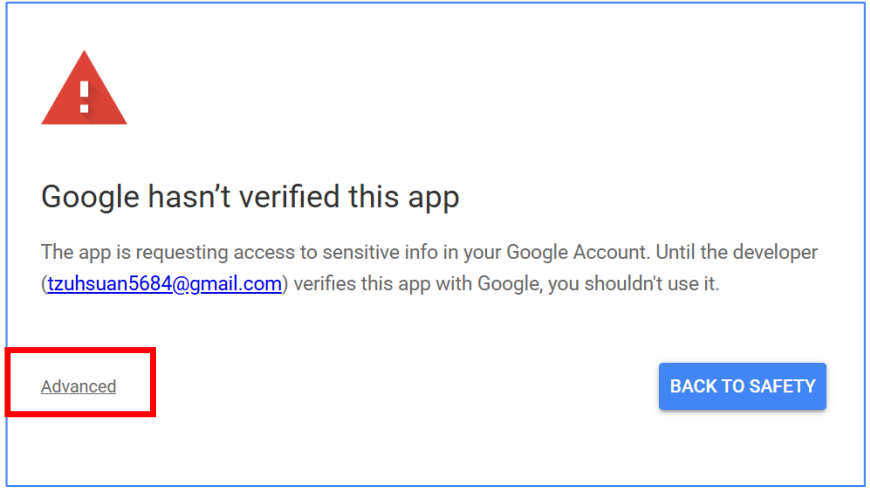
5.部署網頁程式碼

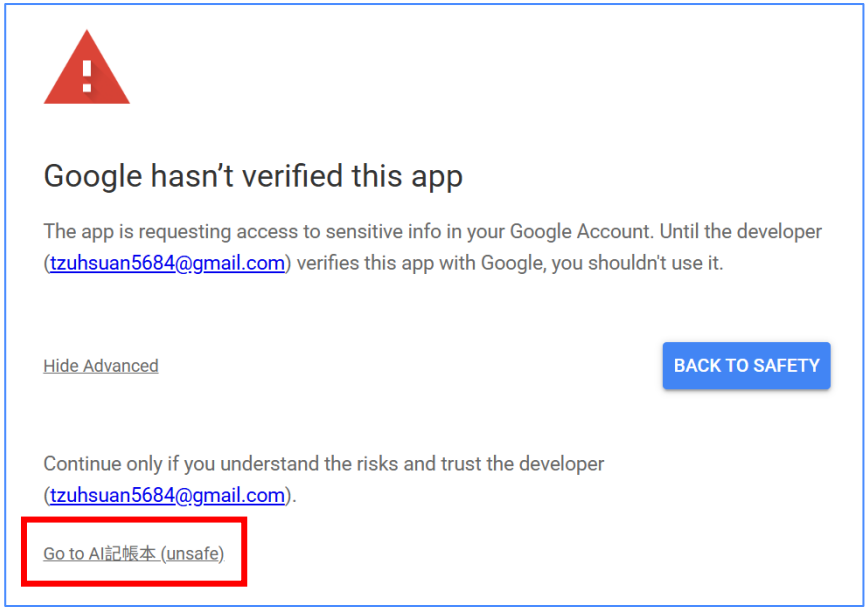
新增部署作業
選取類型
設定
網頁應用程式
API 執行檔
外掛程式
資料庫
請選取部署作業類型
取消
部署

新增部署作業
選取類型
設定
網頁應用程式
說明
新增說明
v1
網頁應用程式
執行身分
我 (tzuhsuan5684@gmail.com)
網頁應用程式將有權在執行時使用您的帳戶資料。
誰可以存取
所有人
這項專案也可以當做程式庫使用。瞭解詳情
取消
部署

5.部署網頁程式碼

- **授權**：點擊部署後，系統會要求授權，請一路點擊「進階」與「前往」允許存取。



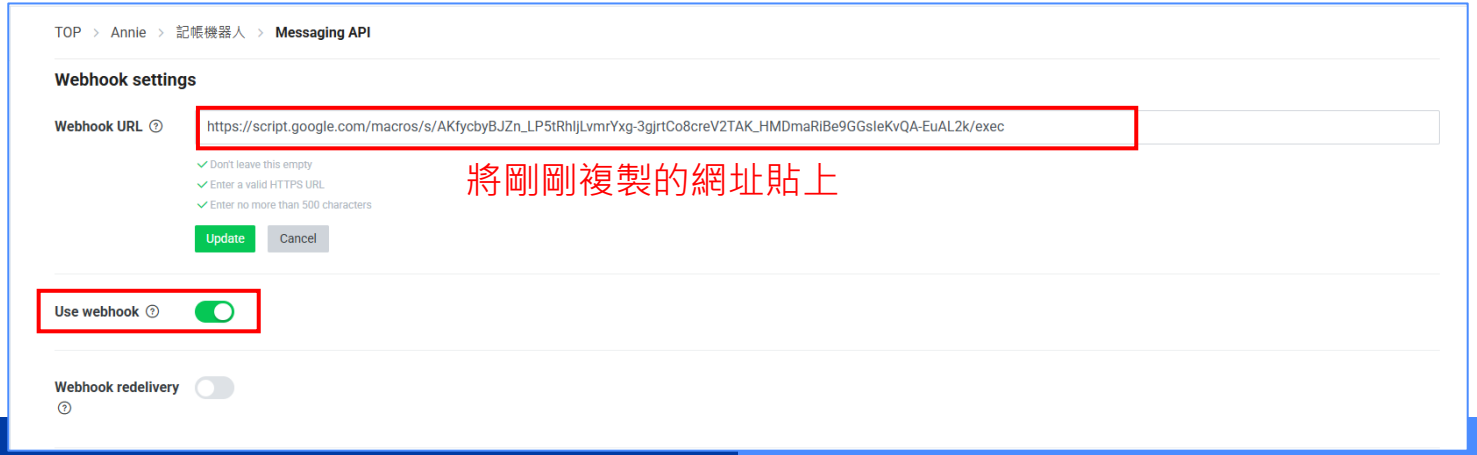


5.部署網頁程式碼

- **複製網址**：複製產生的「**網頁應用程式網址**」。

6. 串接 LINE Webhook

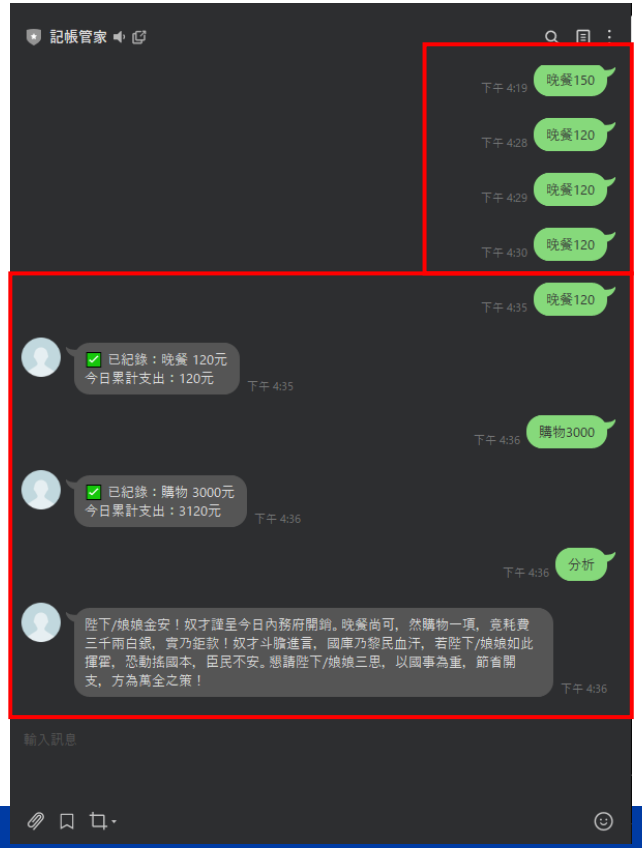
1. 回到 LINE Developers 的 **Messaging API** 頁面。
2. 將剛才複製的網址貼入 **Webhook URL**。
3. 點擊 **Update** 並點擊 **Verify**。
4. 看到綠色的 **Success** 代表大功告成！務必開啟下方的 **Use webhook**。



7. LINE 帳號「回應設定」(關鍵！)

為了讓 AI 能順利回話，必須關閉 LINE 內建的罐頭功能：

測試與除錯



測試:沒反應
除錯:尋找可能原因

Chatbot成功回覆

機器人沒反應？

- 檢查 Use webhook 有沒有打開。
- 回覆設定是否正確設置

機器人回覆「錯誤」？

- 這通常是 API Key 填錯，請檢查 Gemini Key 是否正確無誤。

開始使用

- **一般記帳**：輸入「早餐 100 元」、「買了咖啡 50」。
- **AI 分析**：輸入「分析」或「報告」。

挑戰

試著新增一些新功能吧!

你可以:

- 修改程式碼裡的 prompt 內容，改變 AI 管家的個性
- 新增刪除功能
- 類別標籤
- 視覺化分析